

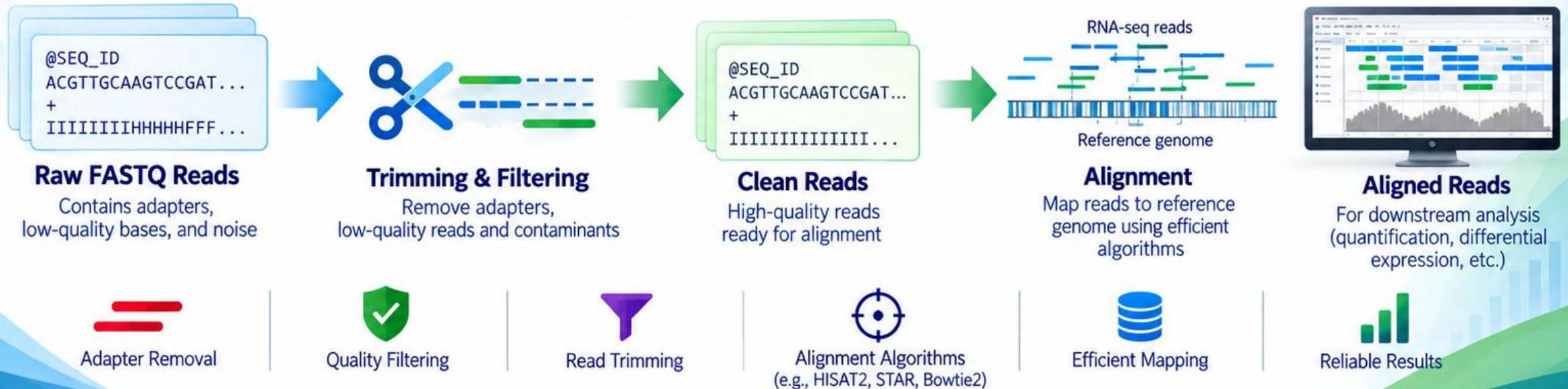
Process Data
Reveal Biology
Drive Discovery



Trimming, Alignment & Alignment Algorithms

From Clean Reads to Accurate Mapping

RNA-seq Workshop | Bioinformatics & Computational Biology



LEARNING OBJECTIVES

By the end of this session, you will be able to:



Understand what **RNA-seq read trimming** is
Learn the concept and purpose of trimming in RNA-seq.



Learn **why raw reads** are trimmed
Understand the sources of low-quality bases, adapter sequences and other unwanted reads.



Know the difference between **adapter trimming** and **quality trimming**
Distinguish the goals and methods of each approach.



Understand **how trimming algorithms** identify **unwanted bases**
Learn the underlying algorithms and detection strategies.



Explore major **trimming strategies and algorithms**
Learn commonly used methods and tools.



Learn **important trimming parameters**
Understand key parameters and their impact on results.



Learn how to **select an appropriate trimming tool**
Compare different tools and choose the best option for your data.



Understand how **trimming affects alignment and downstream analysis**
Learn the impact on mapping, transcript quantification and differential expression.



Learn how to **evaluate trimming results**
Use QC reports and key metrics to assess the quality of trimmed reads.

RNA-seq Reads (Raw FASTQ)

```
@SEQ_ID
AGATCGGAAGAGCCTTGAA...
IIIIIIHHHHIIIIIIII...
```



Read Trimming

- Remove adapters
- Trim low-quality bases
- Filter short reads

Trimmed Reads (Clean FASTQ)

```
@SEQ_ID
GATCGGAAGCCTTGAA...
IIIIIIHHHHIIIIII...
```

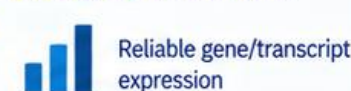


Clean Reads
Better Analysis
Deeper Insights

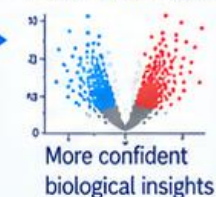
Improved Alignment



Accurate Quantification

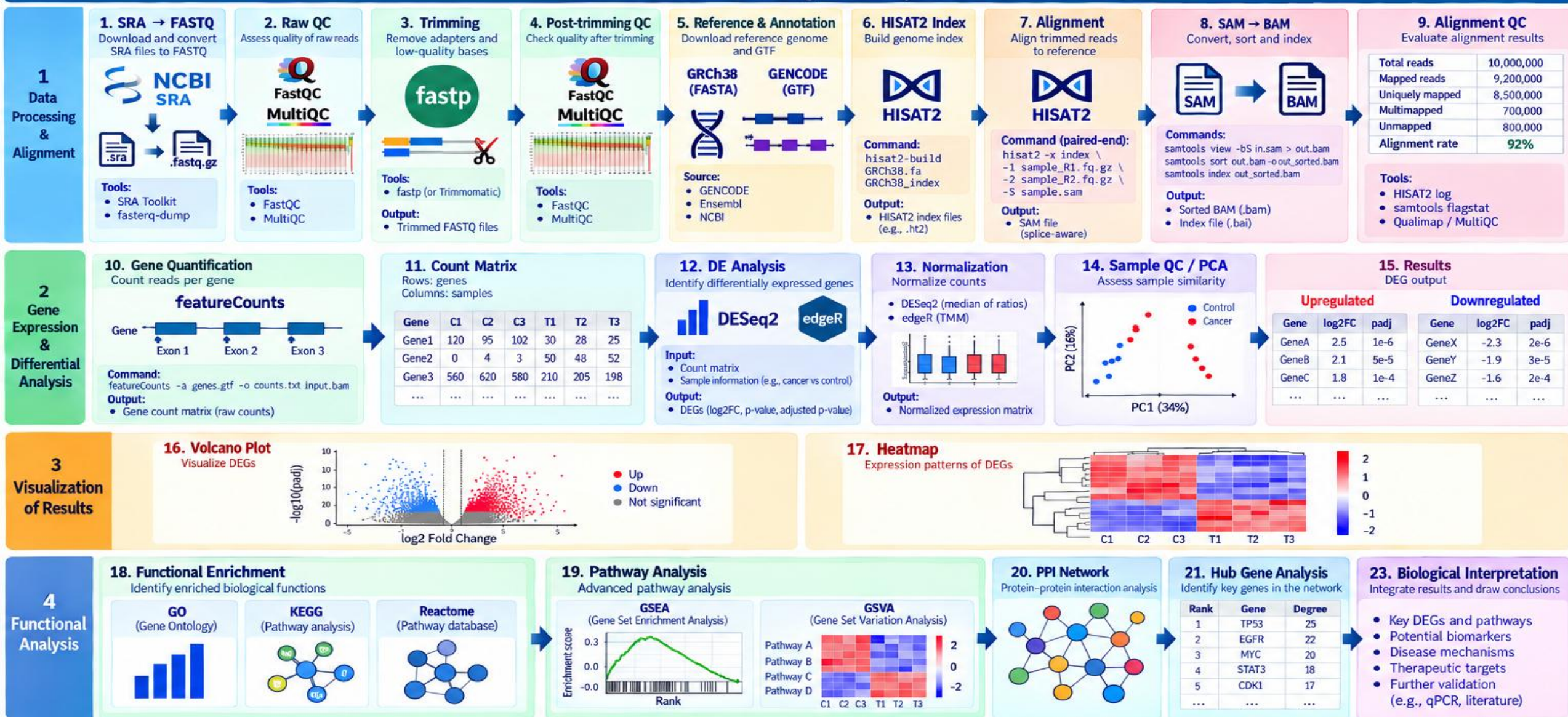


Better Downstream Analysis



Bulk RNA-seq Analysis Workflow (LUAD: 3 Cancer + 3 Control)

From raw data to biological interpretation



Final Output: Comprehensive understanding of molecular mechanisms in LUAD

Why Do Raw RNA-seq Reads Need Trimming?

Remove unwanted sequences • Improve alignment • Ensure accurate quantification

1 Raw RNA-seq Reads May Contain



Sequencing adapters

Adapter sequences from library preparation



Low-quality bases

Bases with low Phred quality scores (e.g. $Q < 20$)



Poor-quality terminal regions

Lower quality at 5' and 3' ends



Technical sequences

Primers, linkers, barcodes, UMIs



Poly-G / poly-A artifacts

Sequencing or library preparation artifacts (e.g. two-color sequencing)



Short or unreliable regions

Very short reads or low-complexity sequences



Other artifacts

PCR duplicates, chimeric reads, contaminants (rRNA, spike-ins), index hopping

2 Sources of Unwanted Sequences

Adapter sequences (e.g. Illumina)



Low-quality bases ($Q < 20$)



Poor terminal quality



Technical sequences (primers / barcodes / UMIs)



Poly-G artifacts (two-color sequencing)



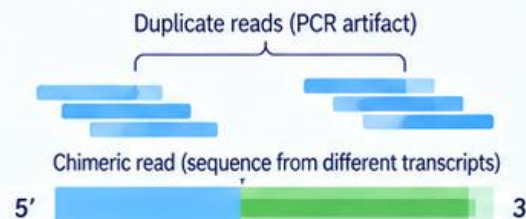
Poly-A/T artifacts (library preparation)



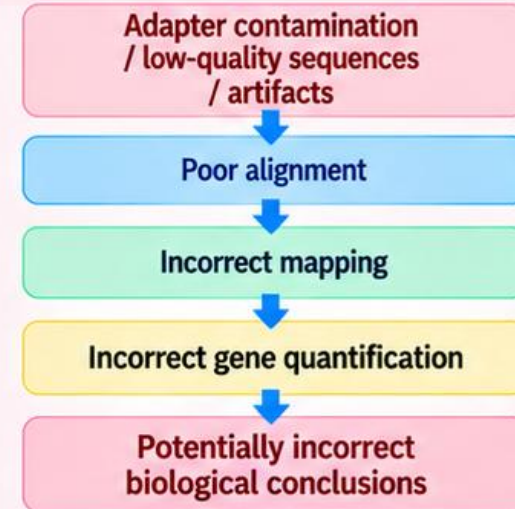
Low-complexity sequences



PCR duplicates / chimeras (artifacts)



3 The Problem



Untrimmed reads can reduce mapping accuracy and lead to erroneous results

4 Common Sources, Examples and Impact

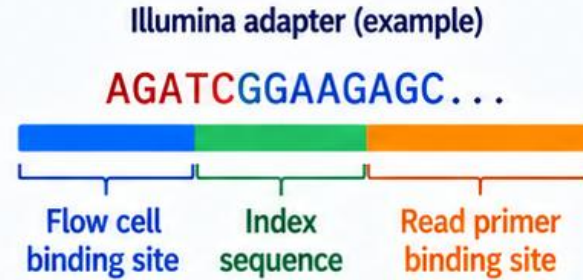
Source	Example	Impact if not trimmed
Adapter	Illumina adapter (AGATCGGAAGAGC...)	False alignment, lower mapping rate
Low-quality bases	$Q < 20$	Increased mismatches, poor alignment
Poor terminal quality	Low Q at 5' / 3' ends	Errors in mapping
Technical sequences	Primers, barcodes, UMIs	Misalignment, incorrect quantification
Poly-G artifacts	GGGGGGG...	False alignments (two-color)
Poly-A/T artifacts	AAAAAAAA...	Incorrect mapping
PCR artifacts	Duplicate reads	Biased expression estimates
Chimeric reads	Sequences from different genes	Mis-mapping
Short/low-complexity	ATATATAT...	Unreliable alignments
Contaminants	rRNA, spike-ins, index hopping	Spurious alignments

Adapter Contamination

Adapters are **artificial sequences** added during library preparation.

Why adapters are added?

- Facilitate sequencing on the platform
- Allow binding to the flow cell
- Enable amplification (PCR)
- Include indices (barcodes) for multiplexing



How adapter contamination occurs?

- The biological RNA fragment (insert) is often **shorter than the sequencing read length**.
- During sequencing, the instrument may read beyond the insert and continue into the adapter.
- As a result, **adapter sequences appear** at the 3' end (or sometimes 5' end) of the read.

Sequenced read (before trimming)



Adapters are necessary for sequencing, but they are not part of the biological sample, so they **must be removed**.

Example of Adapter Contamination

Raw RNA-seq read (before trimming)



After adapter trimming



Trimming Concept

Biological sequence + Adapter sequence



Trimming



Biological sequence (clean read)



Removing adapter sequences prevents **false alignments**, **incorrect mapping** and improves **downstream analysis**.

Types of Trimming in RNA-seq Data

Different trimming strategies are used to remove unwanted sequences and improve read quality.



1 Adapter trimming

Removes library adapters from reads.

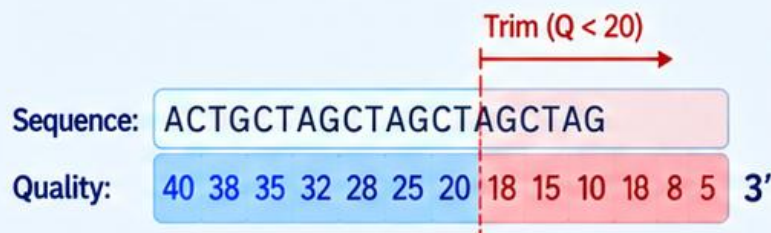


Example:

ACTGCTAGCTAGCTAGCTG AGATCGGAAGAGC...

2 Quality trimming

Removes low-quality bases based on Phred quality scores.



3 End trimming

Removes a fixed number of bases from 5' and/or 3' ends.

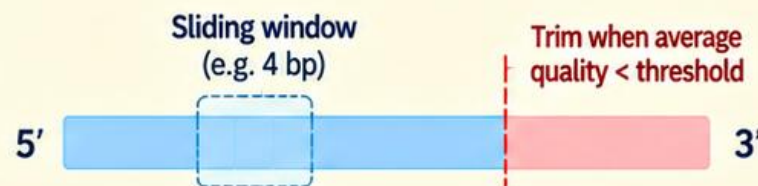


Example: 5 bp from each end

ACTGC TAGCTAGCTA CTAG

4 Sliding-window trimming

Evaluates average quality across a moving window and trims when it falls below a threshold.



5 Minimum-length filtering

Removes reads that become too short (after trimming).



Example: Reads shorter than 20 bp are removed.

6 Poly-X trimming

Removes homopolymer artifacts (e.g. poly-A, poly-T, poly-G, poly-C).

Poly-A: ACTGCTAGAAAAAA → ACTGCTAG

Poly-T: ACTGCTAGTTTTTT → ACTGCTAG

Poly-G: ACTGCTAGGGGGG → ACTGCTAG

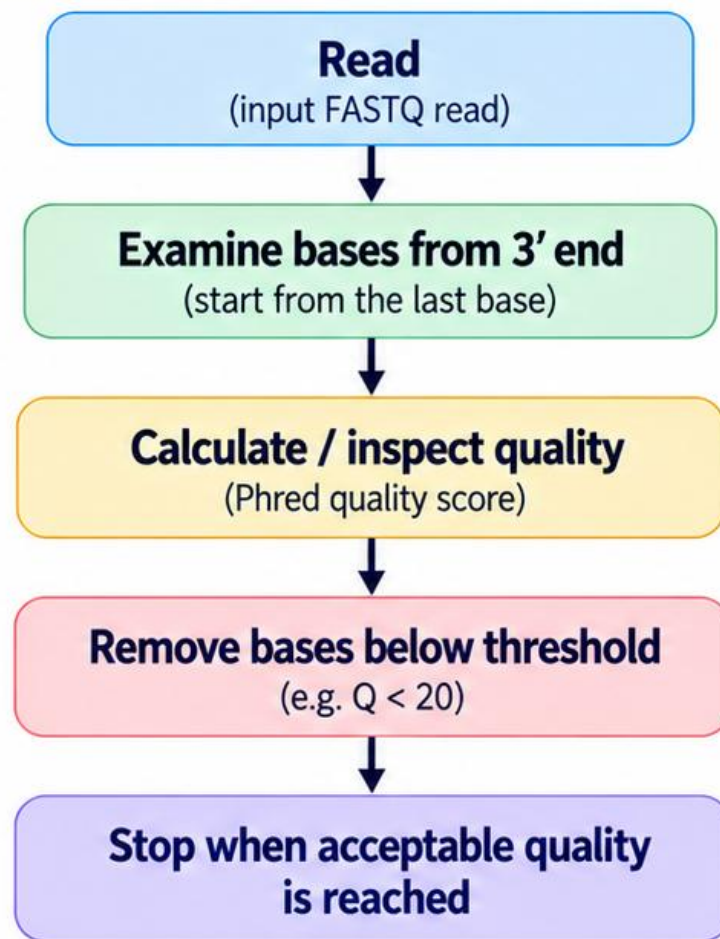
Poly-C: ACTGCTAGCCCCC → ACTGCTAG



Trimming improves read quality by removing adapters, low-quality bases, artifacts and short or unreliable sequences, resulting in **cleaner reads and more accurate downstream analysis**.

Basic Trimming Algorithm

The simplest algorithm:



Example of Basic Trimming

Original read (5' → 3')

A	C	T	G	C	T	A	G	C	T	G	C	T	A	G	C	T	A	G
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Quality scores

40	39	38	37	35	34	33	32	30	29	27	25	23	20	18	15	12	10	8	5
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	---	---

High-quality region
($Q \geq 20$)

Low-quality region
($Q < 20$)

Threshold = Q20

Trimmed read (5' → 3')

A	C	T	G	C	T	A	G	C	T	A	G	C	T	A	G
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Corresponding quality scores

40	39	38	37	35	34	33	32	30	29	27	25	23	20
----	----	----	----	----	----	----	----	----	----	----	----	----	----



Low-quality terminal bases removed



Key idea: Inspect quality from the 3' end and trim bases with quality below a threshold until an acceptable quality is reached.

Result:

Cleaner reads
Better alignment
More accurate downstream analysis

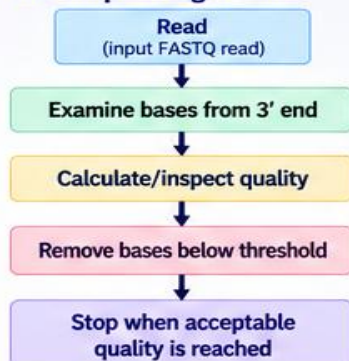
Trimming Algorithms in RNA-seq

Different algorithms are used to remove adapters, low-quality bases and other unwanted sequences to obtain high-quality reads.

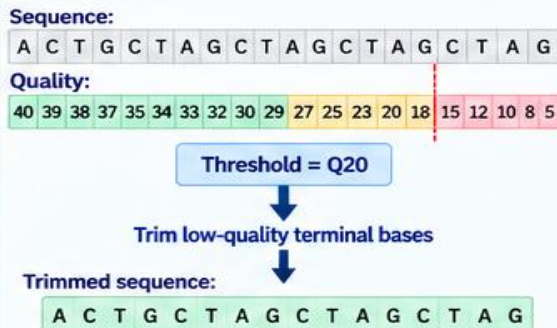


1 Basic Trimming Algorithm

The simplest algorithm:



Example



2 Quality-Based Trimming Algorithm

Basic rule:

IF base_quality < threshold

remove base

Example:

Quality scores:

Q30 Q32 Q31 Q28 Q25 Q18 Q15 Q12

Trim here

Advantages:

- ✓ Simple
- ✓ Fast
- ✓ Easy to interpret

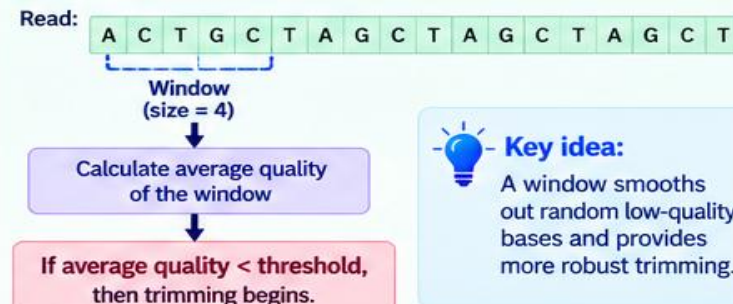
Limitation:

- ✗ Individual low-quality bases may not always indicate an entire poor-quality region.

3 Sliding Window Trimming

Instead of evaluating one base at a time, the algorithm examines a window of bases.

Example (window size = 4):



Key idea: A window smooths out random low-quality bases and provides more robust trimming.

4 Sliding Window Algorithm (Example)

Window size = 4

Quality scores:



Used by:

- Trimmomatic
- Cutadapt (with sliding window option)
- fastp
- Other trimming tools

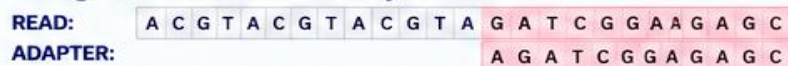
5 Adapter Matching Algorithms

Adapter trimming requires detecting where the adapter begins.

Conceptually:



The algorithm searches for similarity between:



Once detected:

Remove adapter sequence

6 Exact vs Approximate Adapter Matching

Exact matching

Requires an exact sequence match.

READ: A G A T C G G A
ADAPTER: A G A T C G G A

Exact match

Approximate matching

Allows mismatches or sequencing errors.

READ: A G A T C G G A
ADAPTER: A G A T C G G A

More useful with real sequencing data.

7 Seed-and-Extend Concept

More advanced adapter detection can use a seed-and-extend strategy.

1. Seed

Find a short matching region.

READ: ... A G A T C G ...
ADAPTER: ... A G A T C G ...
(seed match)

2. Extend

Extend the match in both directions.

READ: ... A G A T C G G A ...
ADAPTER: ... A G A T C G G A ...

3. Score & Trim

Evaluate matches, mismatches, insertions and deletions. Remove the identified adapter region.



8 Alignment-Based Adapter Trimming

Adapter detection can also be treated as a sequence alignment problem.

Example alignment:

Read: A C G T A C G T A G A T C G G A A G A G C
Adapter: A G A T C G G A G A A G C

Algorithms can use:

- Local alignment
- Edit distance
- Dynamic programming
- Sequence similarity scores

Edit Distance

Measures how many operations are required to transform one sequence into another.

Operations:

- Substitution
- Insertion
- Deletion

Example:

A G A T G G A
A G A T C G A
1 mismatch
Edit distance = 1

Dynamic Programming

Classic sequence alignment algorithms use dynamic programming.

Major concepts:

- Match score
- Mismatch penalty
- Gap penalty

Example scoring matrix (conceptual):

	-	A	G	A	T
-	0	-1	-1	-1	-1
A	-1	1	0	0	0
G	-1	0	1	0	0
A	-1	0	0	2	1
T	-1	0	0	1	3

The optimal alignment identifies the best matching region between the read and adapter.



Key take-home message: Trimming algorithms help remove adapters, low-quality bases and artifacts, resulting in **cleaner reads** and **more accurate downstream analysis**.



Trimmomatic

A flexible read trimming tool for Illumina NGS data

Clean Reads
Better Insights

1. Overview

- Trimmomatic is a widely used tool for trimming Illumina sequencing reads.
- It removes adapter sequences, low-quality bases and other unwanted sequences to produce high-quality clean reads for downstream analysis.
- Developed by the Usadel Lab (University of Tübingen).
- Written in Java.
- Works for both paired-end and single-end reads.
- Highly configurable.



Key Features

- ✓ Removes adapter sequences
- ✓ Trims low-quality bases (5' and 3' ends)
- ✓ Uses sliding-window quality trimming
- ✓ Filters reads by minimum length
- ✓ Suitable for paired-end and single-end data
- ✓ Widely used in RNA-seq analysis

Input and Output



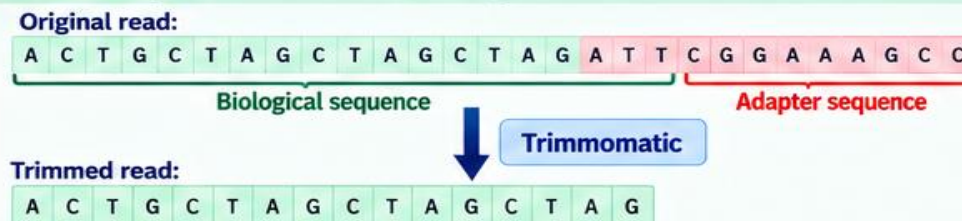
2. Important Operations in Trimmomatic

Operation	Description	Example / Parameters
ILLUMINACLIP	Removes adapter sequences using seed-and-extend matching.	ILLUMINACLIP:adapters.fa:2:30:10 2 = max mismatches in seed 30 = palindrome clip threshold 10 = simple clip threshold
LEADING	Removes low-quality bases from the 5' end.	LEADING:20 Remove bases with Q < 20 from the start of the read.
TRAILING	Removes low-quality bases from the 3' end.	TRAILING:20 Remove bases with Q < 20 from the end of the read.
SLIDINGWINDOW	Performs sliding-window trimming based on average quality.	SLIDINGWINDOW:4:20 Window size = 4 bases Average quality threshold = Q20
MINLEN	Removes reads shorter than a specified length.	MINLEN:36 Discard reads shorter than 36 bp after trimming.

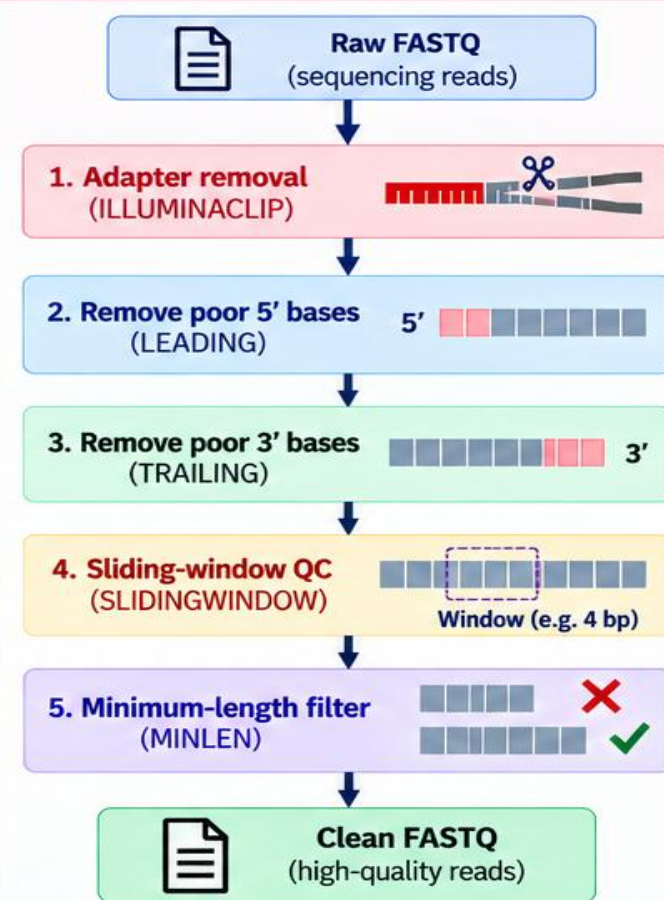
3. Example: SLIDINGWINDOW:4:20



4. Example: Combined Trimming



5. Trimmomatic Workflow



6. Benefits

- ✓ Removes adapters and low-quality bases
- ✓ Improves read quality and mapping accuracy
- ✓ Reduces false alignments and downstream errors
- ✓ Essential preprocessing step for RNA-seq analysis

Trimmomatic – Installation

Prepare the tool
for read trimming

Trimmomatic

A flexible read trimming tool
for Illumina NGS data

- Removes adapter sequences
- Trims low-quality bases
- Works for both paired-end and single-end reads
- Highly configurable
- Widely used in RNA-seq analysis
- Written in Java



Installation (Ubuntu/Linux without Conda)

```
sudo apt update
sudo apt install trimmmmatic
```

Check installation

```
trimmomatic -version
```



If your Linux distribution does not provide the desired version, install it from the official Trimmomatic distribution.

Trimmomatic Parameters

Key options to customize trimming

Parameter	Function	Description	Example
ILLUMINACLIP	Adapter trimming	Removes adapter sequences using seed-and-extend matching.	ILLUMINACLIP:adapters.fa:2:30:10
LEADING	Remove low-quality bases at 5' end	Removes bases with quality < threshold from the start of the read.	LEADING:20
TRAILING	Remove low-quality bases at 3' end	Removes bases with quality < threshold from the end of the read.	TRAILING:20
SLIDINGWINDOW	Sliding window quality trimming	Scans the read with a sliding window and trims when the average quality falls below the threshold.	SLIDINGWINDOW:4:20
MINLEN	Minimum length filter	Discards reads shorter than a specified length after trimming.	MINLEN:36

Example: SLIDINGWINDOW:4:20

- Window size = 4 bases
- Minimum average quality = Q20



Trimmomatic Paired-End Command

Example for RNA-seq data

Input files (paired-end)

Cancer1_R1.fastq.gz } Paired-end reads
Cancer1_R2.fastq.gz }

Output files

R1_paird / R1_unpaired } Trimmed reads
R2_paird / R2_unpaired }

Command

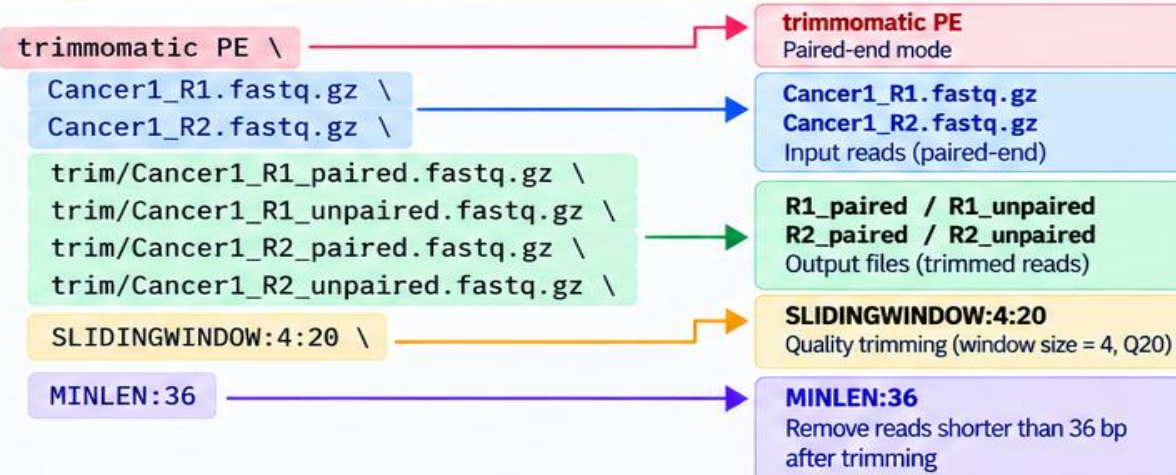
```
trimmomatic PE \  
  Cancer1_R1.fastq.gz \  
  Cancer1_R2.fastq.gz \  
  trim/Cancer1_R1_paird.fastq.gz \  
  trim/Cancer1_R1_unpaired.fastq.gz \  
  trim/Cancer1_R2_paird.fastq.gz \  
  trim/Cancer1_R2_unpaired.fastq.gz \  
  SLIDINGWINDOW:4:20 \  
  MINLEN:36
```



If adapter trimming is required, include an appropriate ILLUMINACLIP setting with the correct adapter FASTA file.

Understanding the Trimmomatic Command

Breakdown of each component



Each parameter can be adjusted based on your data quality and analysis requirements.

Trimming with Trimmomatic

Quality-based trimming for paired-end reads

Clean reads
for accurate
analysis



Trimmomatic



Installation

```
sudo apt install trimmomatic
```

Installs Trimmomatic
from the Ubuntu repository.



Check Version

```
TrimmomaticPE -h
```

Displays help and version
information.

Run Trimmomatic (Paired-end, for all samples)

Cancer1

```
TrimmomaticPE -threads 18 \  
Cancer1_R1.fastq.gz \  
Cancer1_R2.fastq.gz \  
Cancer1_R1_trimmed2.fastq.gz \  
Cancer1_R1_unpaired.fastq.gz \  
Cancer1_R2_trimmed2.fastq.gz \  
Cancer1_R2_unpaired.fastq.gz \  
SLIDINGWINDOW:4:20 \  
MINLEN:36
```

Cancer2

```
TrimmomaticPE -threads 18 \  
Cancer2_R1.fastq.gz \  
Cancer2_R2.fastq.gz \  
Cancer2_R1_trimmed2.fastq.gz \  
Cancer2_R1_unpaired.fastq.gz \  
Cancer2_R2_trimmed2.fastq.gz \  
Cancer2_R2_unpaired.fastq.gz \  
SLIDINGWINDOW:4:20 \  
MINLEN:36
```

Cancer3

```
TrimmomaticPE -threads 18 \  
Cancer2_R1.fastq.gz \  
Cancer2_R2.fastq.gz \  
Cancer2_R1_trimmed2.fastq.gz \  
Cancer2_R1_unpaired.fastq.gz \  
Cancer2_R2_trimmed2.fastq.gz \  
Cancer2_R2_unpaired.fastq.gz \  
SLIDINGWINDOW:4:20 \  
MINLEN:36
```

Control1

```
TrimmomaticPE -threads 18 \  
Control1_R1.fastq.gz \  
Control1_R2.fastq.gz \  
Control1_R1_trimmed2.fastq.gz \  
Control1_R1_unpaired.fastq.gz \  
Control1_R2_trimmed2.fastq.gz \  
Control1_R2_unpaired.fastq.gz \  
SLIDINGWINDOW:4:20 \  
MINLEN:36
```

Control2

```
TrimmomaticPE -threads 18 \  
Control2_R1.fastq.gz \  
Control2_R2.fastq.gz \  
Control2_R1_trimmed2.fastq.gz \  
Control2_R1_unpaired.fastq.gz \  
Control2_R2_trimmed2.fastq.gz \  
Control2_R2_unpaired.fastq.gz \  
SLIDINGWINDOW:4:20 \  
MINLEN:36
```

Control3

```
TrimmomaticPE -threads 18 \  
Control2_R1.fastq.gz \  
Control2_R2.fastq.gz \  
Control2_R1_trimmed2.fastq.gz \  
Control2_R1_unpaired.fastq.gz \  
Control2_R2_trimmed2.fastq.gz \  
Control2_R2_unpaired.fastq.gz \  
SLIDINGWINDOW:4:20 \  
MINLEN:36
```



Trimmomatic Parameters Used

Parameter	Meaning
SLIDINGWINDOW:4:20	Sliding window trimming (window size = 4, average quality ≥ 20)
MINLEN:36	Discard reads shorter than 36 bp after trimming



- Generates paired and unpaired output files
- Uses 18 CPU threads
- Adjust parameters based on data quality

Trimming with fastp

All-in-one FASTQ preprocessing tool

Fast, efficient,
comprehensive
read preprocessing



Installation

```
sudo apt install fastp
```

Installs fastp
from the Ubuntu repository.



Check Version

```
fastp -v
```

Displays the installed
fastp version.

Run fastp (for all samples)

Cancer1

```
fastp -i Cancer1_R1.fastq.gz \  
-I Cancer1_R2.fastq.gz \  
-o Cancer1_R1_trimmed2.fastq.gz \  
-O Cancer1_R2_trimmed2.fastq.gz \  
-q 20 -l 36 \  
-h Cancer1_fastp.html \  
-j Cancer1_fastp.json
```

Cancer2

```
fastp -i Cancer2_R1.fastq.gz \  
-I Cancer2_R2.fastq.gz \  
-o Cancer2_R1_trimmed2.fastq.gz \  
-O Cancer2_R2_trimmed2.fastq.gz \  
-q 20 -l 36 \  
-h Cancer2_fastp.html \  
-j Cancer2_fastp.json
```

Cancer3

```
fastp -i Cancer3_R1.fastq.gz \  
-I Cancer3_R2.fastq.gz \  
-o Cancer3_R1_trimmed2.fastq.gz \  
-O Cancer3_R2_trimmed2.fastq.gz \  
-q 20 -l 36 \  
-h Cancer3_fastp.html \  
-j Cancer3_fastp.json
```

Control1

```
fastp -i Control1_R1.fastq.gz \  
-I Control1_R1.fastq.gz \  
-o Control1_R1_trimmed2.fastq.gz \  
-O Control1_R2_trimmed2.fastq.gz \  
-q 20 -l 36 \  
-h Control1_fastp.html \  
-j Control1_fastp.json
```

Control2

```
fastp -i Control2_R1.fastq.gz \  
-I Control2_R1.fastq.gz \  
-o Control2_R1_trimmed2.fastq.gz \  
-O Control2_R2_trimmed2.fastq.gz \  
-q 20 -l 36 \  
-h Control2_fastp.html \  
-j Control2_fastp.json
```

Control3

```
fastp -i Control3_R1.fastq.gz \  
-I Control3_R1.fastq.gz \  
-o Control3_R1_trimmed2.fastq.gz \  
-O Control3_R2_trimmed2.fastq.gz \  
-q 20 -l 36 \  
-h Control3_fastp.html \  
-j Control3_fastp.json
```



fastp Parameters Used

Parameter	Meaning
-q 20	Quality trimming (Q20)
-l 36	Minimum read length = 36 bp
-h	Generate HTML report
-j	Generate JSON report



- Performs quality filtering, adapter trimming and reporting
- Generates trimmed FASTQ files, HTML and JSON reports
- Fast and memory efficient
- Adjust parameters based on your data and requirements

What Happens After Trimming?

From clean reads to aligned reads

Step by step
towards biological
insights

Pre-alignment: From Raw Reads to Clean Reads

Quality control and trimming to remove low-quality bases and adapters



Raw FASTQ

(Sequencing output)

- Contains adapters
- Low-quality bases
- Possible artifacts



Quality Control

- Assess read quality (e.g., FastQC)
- Identify adapters and low-quality regions
- Generate QC reports



Trimming

- Remove adapters
- Trim low-quality bases
- Filter short reads (e.g., Trimmomatic, fastp)



Clean FASTQ

- High-quality reads
- Adapters removed
- Ready for alignment

Post-trimming: From Clean Reads to Aligned Reads

Align clean reads to a reference genome to generate SAM/BAM files



Clean FASTQ

- Trimmed and high-quality reads
- Input for alignment



Reference Genome

- e.g., Human GRCh38
- Includes genome sequence and annotation
- Used by aligners for mapping



Alignment / Mapping

- Map reads to reference genome
- Splice-aware alignment (for RNA-seq)
- Tools: HISAT2, STAR, etc.



SAM/BAM

- SAM: text format (alignment information)
- BAM: binary format (compressed)
- Contains mapped read information (chromosome, position, CIGAR, etc.)



Clean reads are aligned to a reference genome to generate SAM/BAM files for downstream analysis

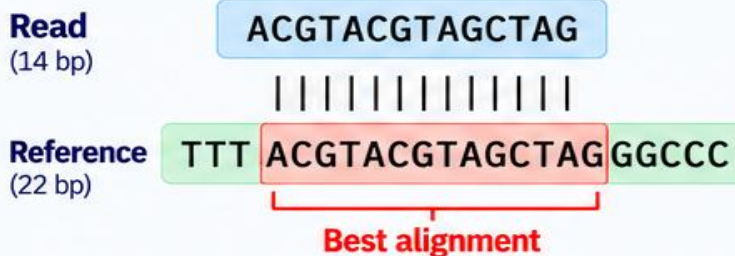
From reads to results

What is Read Alignment?

Finding the position where a read best matches the reference

Read alignment is the process of finding the position in a reference sequence where a sequencing read best matches.

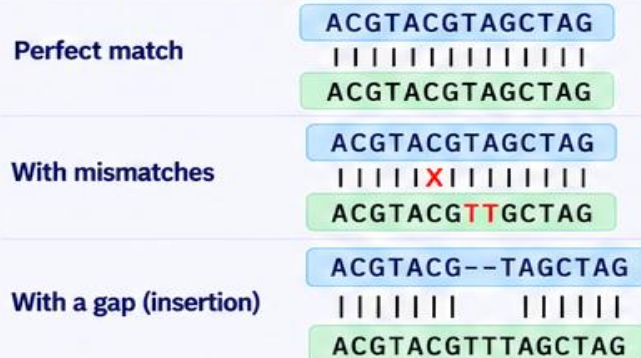
Example:



The aligner determines:

- Where the read originated
- How well it matches
- Whether mismatches exist
- Whether gaps are required (insertions/deletions)

Other possible alignments

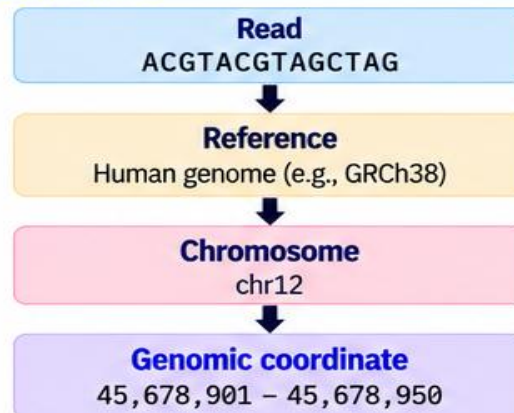


What is Mapping?

Assigning a read to a location in the reference genome

Mapping means assigning a sequencing read to a location in the reference genome/transcriptome.

Mapping workflow



Example:



Alignment vs Mapping

Feature	Mapping	Alignment
Meaning	Locating the read in the reference	Determining the base-by-base correspondence
Focus	Where the read falls	How the read matches
Output	Chromosome and coordinate	Aligned sequence with matches/mismatches/gaps
Relationship	Often used interchangeably	A more detailed process within mapping



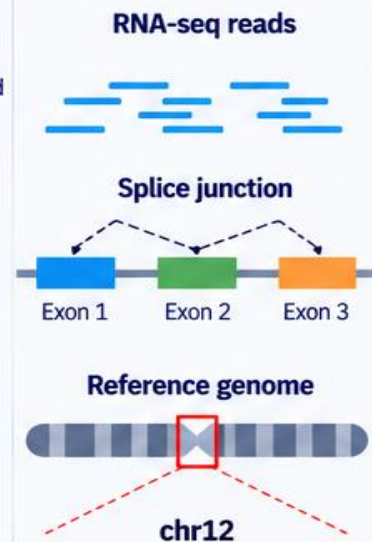
Mapping = locating the read
Alignment = determining the base-by-base correspondence

Why Do We Align RNA-seq Reads?

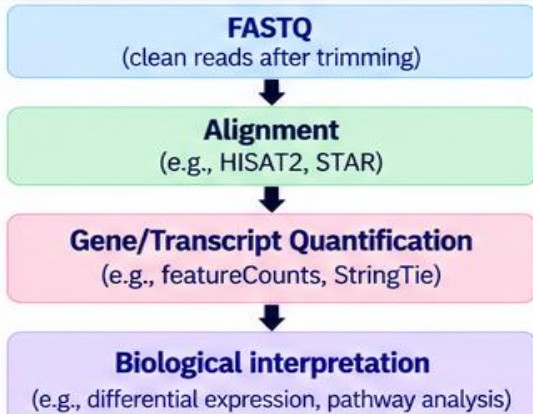
Alignment enables us to extract biological information

Alignment allows us to determine:

- Which genes are expressed
- Which transcripts are expressed
- Where reads originated
- Exon usage
- Splice junctions
- Alternative splicing
- SNPs/variants
- Transcript structure



Overall Workflow and Goal



Ultimately, alignment of RNA-seq reads helps us understand gene regulation and biological processes in health and disease.



Why RNA-seq Alignment is Special?

RNA-seq reads span exon-exon junctions

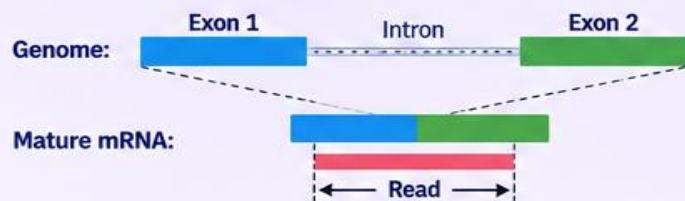
DNA-seq

- Reads generally originate from continuous genomic DNA.
- Aligners can map reads directly to the genome.



RNA-seq

- RNA is processed by splicing (introns are removed).
- Reads may span exon-exon junctions.

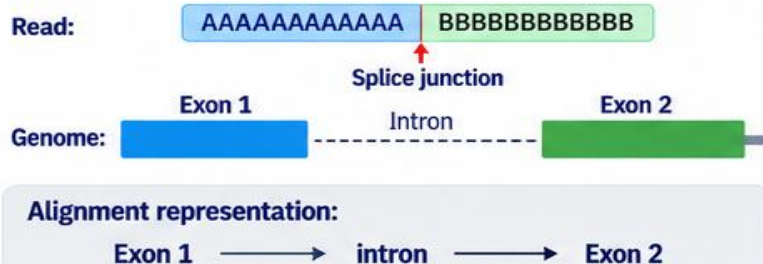


Therefore, RNA-seq requires **splice-aware alignment**.

What is Splice-Aware Alignment?

Recognizing reads that span introns

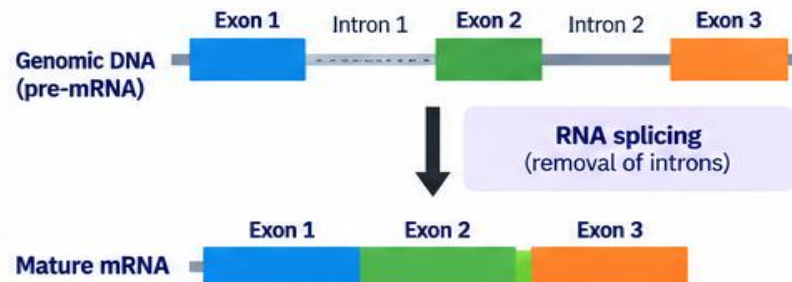
A splice-aware aligner can recognize that a read spans an intron and align the read in a split manner.



The aligner allows for large gaps (introns) and aligns each part of the read to the corresponding exon.

RNA Splicing

Introns are removed to generate mature mRNA



RNA-seq reads may span exon-exon junctions

Read within exon



Maps entirely within an exon

Junction-spanning read



Spans exon 1 and exon 2 (even though separated by a large intron in genome)

Read within exon



Maps entirely within an exon

Canonical Splice Junctions

Most eukaryotic introns have characteristic boundaries

5' splice site (donor site)

GT

3' splice site (acceptor site)

AG

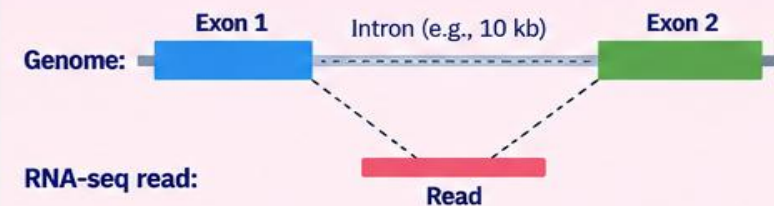
- The most common canonical splice junction pattern is 5'-GT ... AG-3'.
- This information is used by aligners to improve alignment accuracy.
- Other non-canonical splice patterns (e.g., GC-AG, AT-AC) can also occur, so aligners should not assume every junction is strictly GT-AG.

Examples of splice site motifs

Canonical (most common)	GT	AG
Minor class	GC	AG
Minor class	AT	AC

The Splice Junction Problem

A read spans two exons that are distant in the genome



- The read contains sequence from Exon 1 + Exon 2 (the intron is absent).
- A standard genomic aligner may fail to align this read or report a poor alignment.
- Therefore, **RNA-seq requires splice-aware alignment**.

Alignment Challenges in RNA-seq

Why it is more complex than DNA-seq

	Exon-exon junctions	Reads may span splice junctions
	Large introns	Introns can be very large (kb to Mb)
	Alternative splicing	Multiple isoforms from the same gene
	Sequencing errors	Errors can lead to mismatches
	SNPs / variants	Natural variation in the genome
	Repetitive sequences	Reads may map to multiple locations
	Multimapping reads	Some reads map to multiple loci
	Low-quality reads	May cause incorrect alignments
	Novel splice junctions	Unknown junctions not in annotation

RNA-seq aligners must handle all these challenges simultaneously to produce accurate and reliable alignments.

Basic Alignment Algorithms

Traditional approaches for sequence alignment

Global Alignment

Aligns entire sequences from end to end. Useful when sequences are of similar length.

Query ACTGACCTGA
Ref ACTGACCTGA

Global alignment
(Needleman–Wunsch)

Local Alignment

Finds the best matching subsequence within longer sequences. Useful when sequences are of different lengths or share only a local region.

Query TTACGTACGTAA
Ref GGACGTACGTTT

Local alignment
(Smith–Waterman)

Algorithmic foundations



Dynamic programming
Optimal alignment using scoring matrix



Seed-and-extend
Find short matching seeds and extend



Burrows-Wheeler Transform
Rearrange genome for efficient searching



FM-index
Compressed index for fast exact matching



Suffix-based indexing
Alternative indexing strategy (e.g., suffix array)

These algorithms form the basis of modern read alignment tools.

Dynamic Programming

Finds the optimal alignment using a scoring matrix

Dynamic programming (DP) computes the best alignment by filling a score matrix based on match, mismatch, insertion and deletion scores.

Example score matrix (conceptual)

	-	A	C	G	T
-	0	-1	-1	-1	-1
A	-1	2	-1	-1	-1
C	-1	-1	2	-1	-1
G	-1	-1	-1	2	-1
T	-1	-1	-1	-1	2

Match = +2 Mismatch = -1 Gap (insertion/deletion) = -1

The algorithm evaluates:

- Match (same base)
- Mismatch (different base)
- Insertion (gap in read)
- Deletion (gap in reference)
- Overall alignment score

Limitation

Full dynamic programming is computationally expensive for whole genomes (billions of bases).

Therefore modern aligners use indexing and seed-based approaches to improve speed and scalability.

Seed-and-Extend Algorithm

A common strategy used by modern aligners

- 1 Input read**
Take a sequencing read
- 2 Find short matching seed**
Extract a short k-mer (seed) from the read
- 3 Locate seed in genome**
Search for exact or near-exact matches
- 4 Extend alignment**
Extend the seed to a full alignment (using heuristics/DP)
- 5 Evaluate alignment score**
Check mismatches, gaps, and score
- 6 Select best location**
Report the best alignment(s)

Example

Read (16 bp)

ACGTACGTAGCTAGCT

Choose a seed (e.g., 7 bp)

ACGTAGC

Search in genome

...TTTACGTAGGCTGA...

Extend alignment

...ACGTACGTAGCTAGCT...

The seed provides a quick way to find candidate locations, which are then extended to form the full alignment.

Genome Indexing

Pre-process the reference genome for fast searching



Reference Genome
(e.g., human GRCh38)

```
>chr1  
ACTGACCTG...  
>chr2  
TTGACCTG...
```



Build Index

- Process the genome sequence
- Create an index data structure
- Store in compressed form



Indexed Genome

- Efficient storage
- Allows fast searching
- Used by aligners (e.g., HISAT2, STAR, BWA)



Fast Searching

- Look up seed/k-mer in index
- Retrieve candidate locations quickly



Read Mapping

- Extend and evaluate alignments

Indexing dramatically improves computational efficiency compared to searching the entire genome for every read.

Burrows-Wheeler Transform & FM-index

Efficient indexing structures for short-read alignment

Burrows-Wheeler Transform (BWT)

- Rearranges the genome sequence into a form that groups similar substrings together.
- Enables efficient searching.

FM-index

- A compressed index based on the BWT.
- Allows fast and memory-efficient exact matching of short sequences.
- Widely used in modern aligners (e.g., BWA, Bowtie, HISAT2).

Conceptual workflow



BWT + FM-index allow efficient searching of very large genomes, making whole-genome alignment feasible.

Major RNA-seq Alignment Tools

Popular aligners and their key features

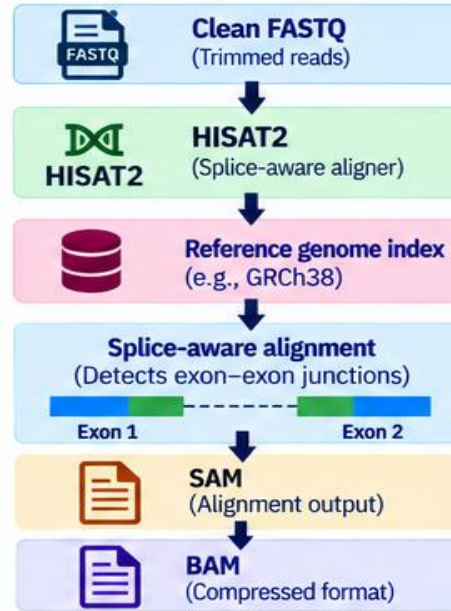
Aligner	Main feature	Remarks
HISAT2	Splice-aware, efficient	Widely used for short-read RNA-seq
STAR	Very fast, high-performance splice alignment	Excellent for large datasets
Bowtie2	General short-read alignment	Not primarily splice-aware
TopHat2	Older splice-aware RNA-seq aligner	Superseded by HISAT2/STAR
minimap2	Particularly useful for long reads	Also works for other sequencing applications



For your project: **HISAT2** is a suitable choice for conventional short-read paired-end RNA-seq.

HISAT2 Concept

A fast and splice-aware RNA-seq aligner



Key points:

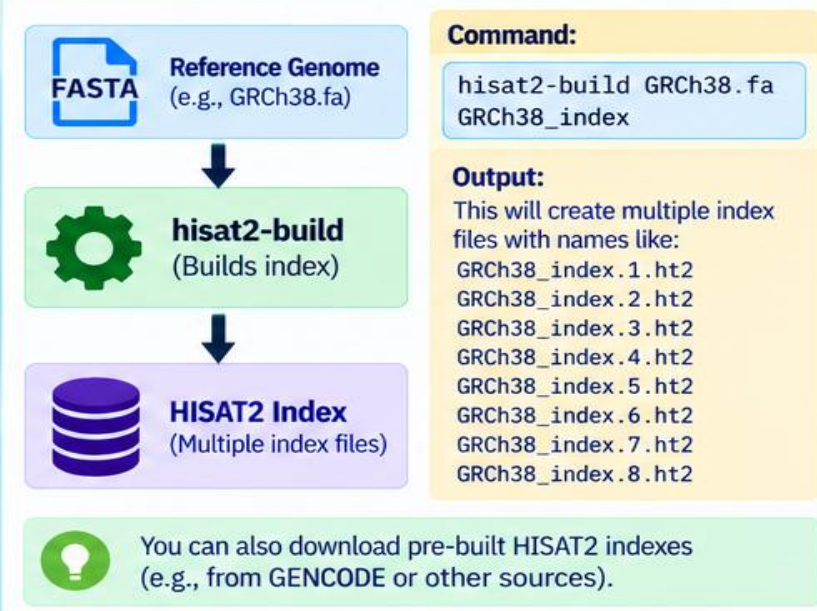
- Splice-aware alignment
- Uses hierarchical indexing (FM-index)
- Efficient for large genomes
- Handles exon-exon junctions
- Suitable for paired-end RNA-seq

Designed to:

- Align reads to a genome
- Account for splicing
- Be fast and memory efficient

HISAT2 Genome Index

Build an index from the reference genome



Command:

```
hisat2-build GRCh38.fa GRCh38_index
```

Output:

This will create multiple index files with names like:
GRCh38_index.1.ht2
GRCh38_index.2.ht2
GRCh38_index.3.ht2
GRCh38_index.4.ht2
GRCh38_index.5.ht2
GRCh38_index.6.ht2
GRCh38_index.7.ht2
GRCh38_index.8.ht2



You can also download pre-built HISAT2 indexes (e.g., from GENCODE or other sources).

HISAT2 Alignment Command

Align paired-end RNA-seq reads to the reference genome

Command (one sample):

```
hisat2 -p 18 \
  -x GRCh38_index \
  -1 Cancer1_R1_trimmed2.fastq.gz \
  -2 Cancer1_R2_trimmed2.fastq.gz \
  -S Cancer1.sam
```

Parameter description:

Parameter	Meaning
-p 18	Use 18 threads
-x GRCh38_index	Reference genome index
-1	Input file for read 1 (R1)
-2	Input file for read 2 (R2)
-S	Output SAM file



This aligns the trimmed paired-end reads to the reference genome and generates a **SAM** file.

HISAT2 Alignment with Log File

Save alignment statistics to a log file

Command (with log file):

```
hisat2 -p 18 \
  -x GRCh38_index \
  -1 Cancer1_R1_trimmed2.fastq.gz \
  -2 Cancer1_R2_trimmed2.fastq.gz \
  -S Cancer1.sam
2> Cancer1_hisat2.log
```

The log file contains useful information:

- Total reads
- Overall alignment rate
- Concordant alignment rate (paired-end)
- Discordant alignment rate
- Unaligned reads
- Summary of alignment statistics



Each sample will generate a separate log file (e.g., Cancer1_hisat2.log).

SAM Output

Sequence Alignment/Map format

SAM (Sequence Alignment/Map) is a text-based format that stores alignment information.

Key fields in a SAM file:

1. Read name (QNAME)
2. Alignment flag (FLAG)
3. Chromosome (RNAME)
4. Position (POS)
5. Mapping quality (MAPQ)
6. CIGAR string (CIGAR)
7. Mate information (RNEXT, PNEXT, TLEN)
8. Sequence (SEQ)
9. Quality (QUAL)
10. Optional fields (e.g., NM, XS, ...)

Example (simplified):

Read001	99	chr1	12345	60	75M	...
↓	↓	↓	↓	↓	↓	
Read name	Flag	Chromosome	Position	MapQ	CIGAR string	

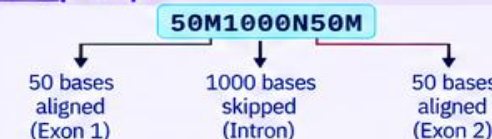
CIGAR String

Describes how the read aligns to the reference

Common CIGAR operations:

Code	Meaning	Description
M	Match	Aligned base (match or mismatch)
I	Insertion	Insertion to the reference
D	Deletion	Deletion from the reference
N	Skipped region	Skipped region (intron, for RNA-seq)
S	Soft clipping	Clipped sequence (not aligned)
H	Hard clipping	Clipped sequence (not present in SEQ)
=	Sequence match	Exact match (optional)
X	Sequence mismatch	Mismatch (optional)

RNA-seq example:



The "N" operation is crucial for RNA-seq, as it represents introns between exons, enabling splice-aware alignment.

SAM → BAM

Convert text-based SAM files to compressed BAM files

- SAM files are text-based and human-readable.
- They contain alignment information for each read.
- However, SAM files are very large.
- Therefore, they are commonly converted to BAM (Binary Alignment/Map), which is a compressed binary format.

SAM



SAM

(text-based)

```
Read001 99 chr1 ...
Read002 147 chr1 ...
Read003 83 chr2 ...
...
```

samtools



BAM



BAM

(binary, compressed)

```
0100101101001010
0101010010101011
0100101001010100
...
```

Command

```
samtools view -@ 18 -b Cancer1.sam > Cancer1.bam
```

Parameters

Parameter	Meaning
-@ 18	Use 18 threads
-b	Output in BAM format (binary)
Cancer1.sam	Input SAM file
> Cancer1.bam	Output BAM file (redirect)



Advantages of BAM

- Compressed (smaller file size)
- Faster to read and write
- Widely used in downstream analysis tools

BAM Sorting

Organize alignments by genomic coordinate

Sort the BAM file

For downstream analysis, BAM files should be sorted by genomic coordinate.

```
samtools sort -@ 18 \
-o Cancer1_sorted.bam \
Cancer1.bam
```

Parameters

Parameter	Meaning
-@ 18	Use 18 threads
-o Cancer1_sorted.bam	Output sorted BAM file
Cancer1.bam	Input BAM file

Index the sorted BAM file

After sorting, create an index file:

```
samtools index Cancer1_sorted.bam
```

Output files

This will generate two files:

```
Cancer1_sorted.bam    (sorted BAM file)
Cancer1_sorted.bam.bai (index file)
```



Best Practice

- Always sort and index BAM files after alignment.
- Most downstream tools (e.g., variant calling, visualization, quantification) require sorted and indexed BAM files.
- Keep both .bam and .bai files in the same directory.

Why Sort and Index BAM?

Essential steps for downstream analysis

Sorting: Organizes alignments by genomic coordinate

- Sorting arranges the reads in order of chromosome and position.
- This makes the file more organized and efficient for subsequent analyses.
- Many tools require coordinate-sorted BAM files.

Example (unsorted vs. sorted)

Unsorted BAM (random order)

```
chr2 500
chr1 300
chr1 100
chr2 200
chr1 400
...
```



Sorted BAM (by coordinate)

```
chr1 100
chr1 200
chr1 300
chr1 400
chr2 100
chr2 200
...
```

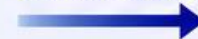
Indexing: Enables rapid access to genomic regions

- Indexing creates a separate index file (.bai) that allows quick retrieval of alignments from specific genomic regions.
- This is essential for:
 - Genome browsers (e.g., IGV)
 - Variant calling
 - Read counting and quantification
 - Other downstream analysis tools



Sorted BAM
(Cancer1_sorted.bam)

samtools index



Index file
(Cancer1_sorted.bam.bai)



Key Takeaway

Sorting and indexing BAM files improve efficiency, enable random access to genomic regions, and are required by most downstream tools.

Mapping Quality (MAPQ)

Represents the confidence that a read is correctly mapped

- MAPQ is a score that estimates the probability that a read is correctly mapped to a genomic location.
- Higher MAPQ → more confident alignment.
- Lower MAPQ → less confident alignment.
- MAPQ is **not the same as the FASTQ Phred base-quality score** (which measures the quality of individual bases).

MAPQ score (typical range 0–60)



Low MAPQ (0–10)

- Less confident alignment
- Often multimapping reads
- Common in repetitive regions

High MAPQ (≥30)

- More confident alignment
- Likely unique mapping
- Higher reliability for downstream analysis

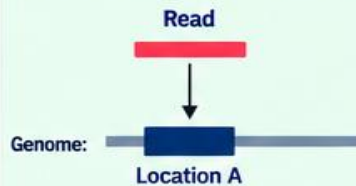
MAPQ reflects mapping confidence, **not base-call quality**.

Unique vs Multimapping Reads

Reads may map to one or multiple genomic locations

Unique mapping read

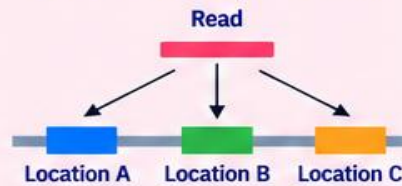
Maps to a single genomic location



Unique mapping
(high MAPQ)

Multimapping read

Maps to multiple genomic locations



Multimapping read
(low MAPQ)

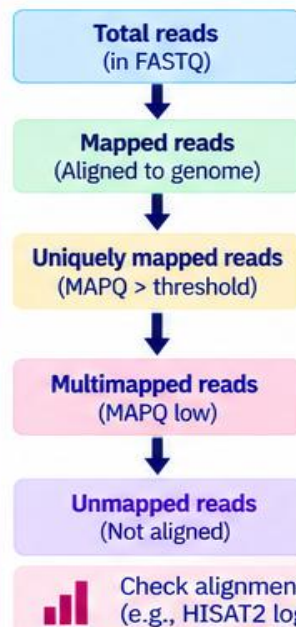
Why do multimapping reads occur?

- Repetitive genomic regions (e.g., transposons, gene families)
- Paralogous genes
- Pseudogenes
- Highly similar sequences

Multimapping reads are common and need to be handled appropriately during downstream analysis.

Alignment Rate

Summary of alignment results



Example (from alignment log)

Category	Number of reads
Total reads	10,000,000
Mapped reads	9,200,000
• Uniquely mapped	8,500,000
• Multimapped	700,000
Unmapped reads	800,000

Alignment rate (%) =
(Mapped reads / Total reads) × 100

Example:
(9,200,000 / 10,000,000) × 100 = **92%**



Check alignment statistics in the aligner log file (e.g., HISAT2 log) to evaluate mapping performance.

What Determines Mapping Rate?

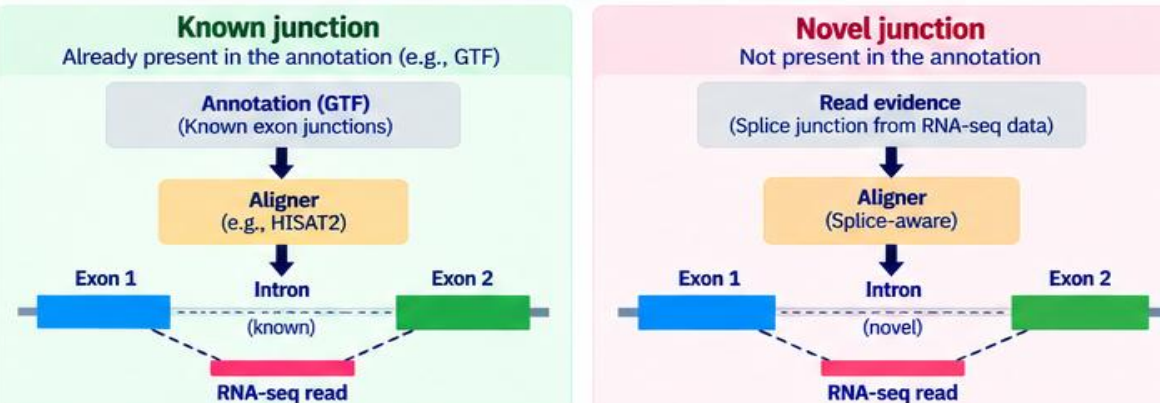
Multiple factors can influence the proportion of mapped reads



A low mapping rate should be investigated (e.g., check quality, contamination, reference), rather than automatically “fixed” by changing parameters.

Known vs Novel Splice Junctions

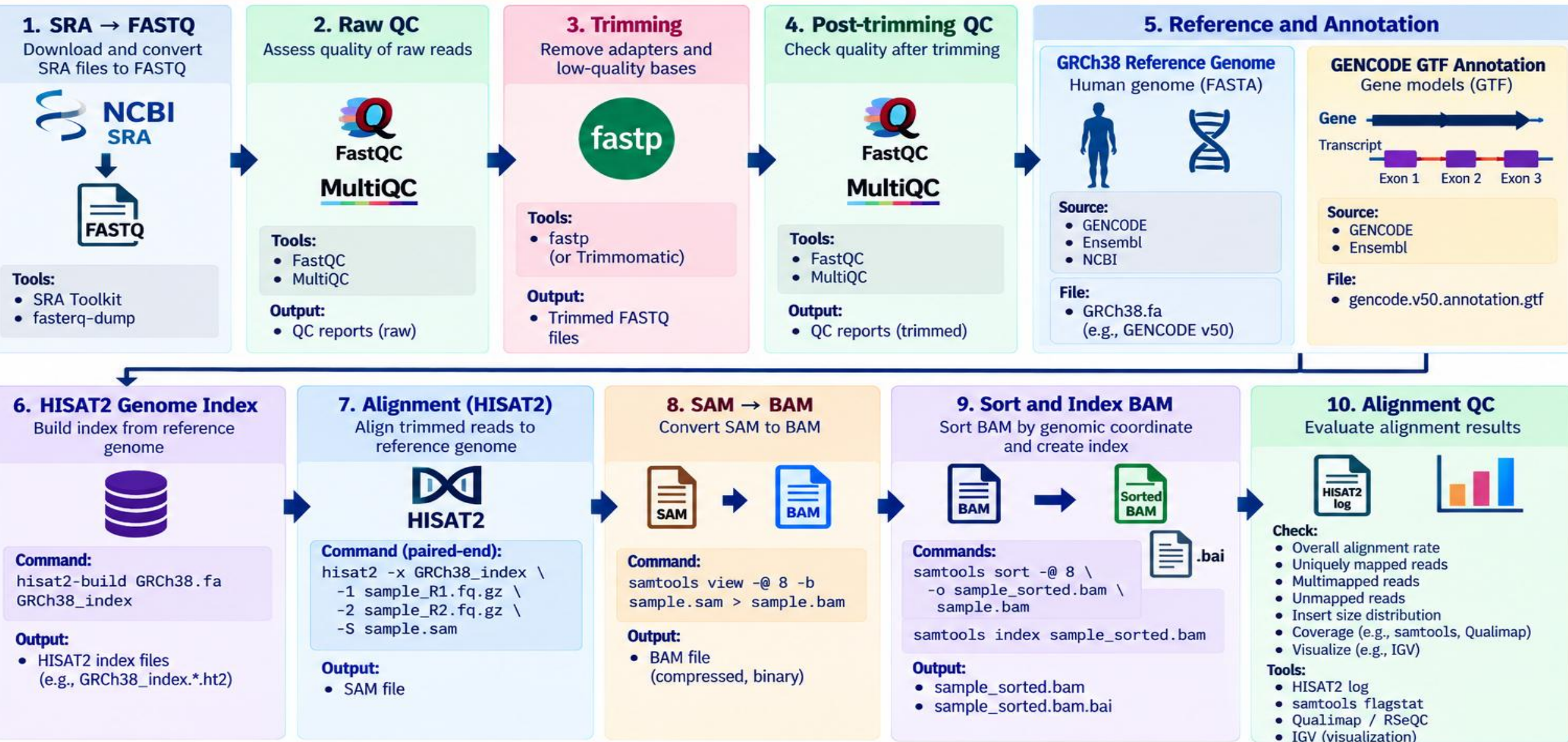
RNA-seq aligners can use known annotations and also discover novel junctions



Modern RNA-seq aligners (e.g., HISAT2, STAR) can use annotation while still allowing discovery of novel splice junctions, depending on the configuration.

RNA-seq Alignment Workflow (LUAD: 3 Cancer + 3 Control)

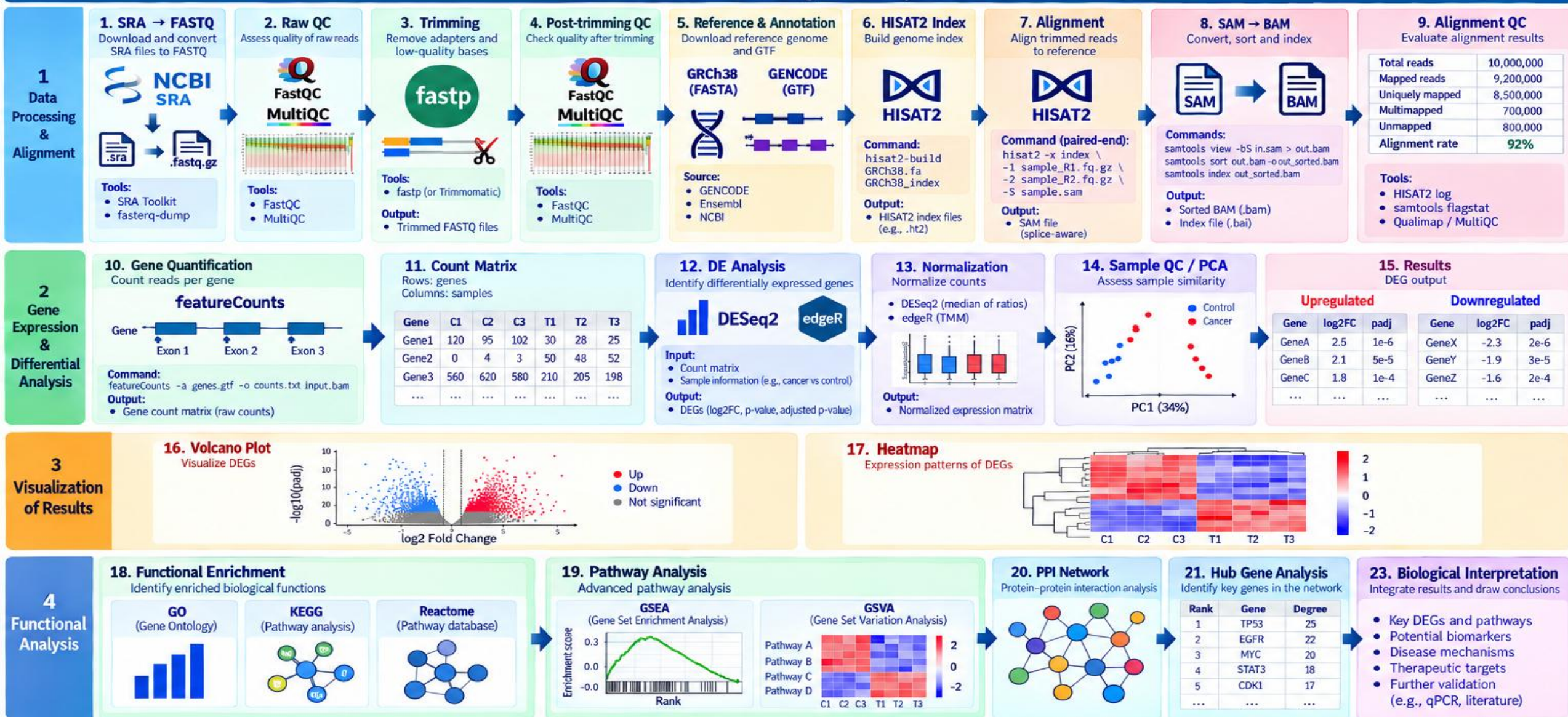
From SRA download to aligned and indexed BAM files with quality control



Result: High-quality, aligned, sorted, and indexed BAM files ready for downstream analysis (e.g., gene counting and differential expression).

Bulk RNA-seq Analysis Workflow (LUAD: 3 Cancer + 3 Control)

From raw data to biological interpretation



Final Output: Comprehensive understanding of molecular mechanisms in LUAD